

Bridging the Semantic Gap in Process Mining

Manufactured Intelligence and Compile-Time Jurisdictions

Sean Chatman

*A dissertation submitted in partial fulfillment
of the requirements for the degree of*
Doctor of Philosophy

Department of Computer Science

June 2026

Abstract

Process mining resides at the critical intersection of data science, formal methods, and operational intelligence. Historically, its application has been severely constrained by the epistemological fragility of dynamic interpreter states.

This dissertation rejects the premise of downstream runtime evaluation. Instead, we propose a paradigm of *Manufactured Intelligence*: intelligence constructed not through AI generation, but through rigid protocol constraints and compile-time jurisdictions.

We present a unified stack: `wasm4pm` acts as the deterministic execution authority; `tower-lsp-max` serves as the protocol bridge; and `pm4py-lsp` functions as a living constraint surface. Crucially, we introduce `wasm4pm-compatible` not as a library, but as a *type-law court*. By forcing downstream dependencies onto Nightly Rust, applications do not domesticate the library; they conform upward to its future type law. We demonstrate that `pm4py-lsp` is a process-intelligence intake membrane. By catching invalid intent upstream, classifying admissibility via typestate inference, and routing valid intent through strict boundary stages (`formats`, `strict`, `wasm4pm`), we manufacture replayable, receipt-backed evidence.

Acknowledgments

This work is the culmination of years of iterative engineering, architectural refactoring, and deep theoretical exploration. I owe a profound debt to the open-source communities surrounding the Rust programming language, WebAssembly, and PM4Py.

The architecture of **tower-lsp-max** stands on the shoulders of the **tower** ecosystem. I extend my deepest appreciation to the adversarial review agents who dismantled early iterations of this pipeline. Their unrelenting demand to move beyond "better PM4Py tooling" and into the realm of a *language-to-process manufacturing membrane* and a *compile-time jurisdiction* drove this research to its absolute physical limits.

Contents

1	Introduction	1
1.1	The Blue River Dam	1
1.2	Compile-Time Jurisdictions	1
1.3	The Unbeknownst Paradigm	1
2	Background and Literature Review	3
2.1	Process Mining Topologies	3
2.2	The WebAssembly Execution Authority	3
2.3	The Language Server Protocol (LSP) as a Bridge	3
3	The Deterministic Execution Authority: WASM4PM	5
3.1	The Court of Admissibility	5
3.2	Columnar Substrates and Zero-Copy Parsing	5
3.3	Algorithmic Deconstruction: Discovery	5
4	The Protocol Substrate: Tower-LSP-Max	7
4.1	The Language/Protocol Bridge	7
4.2	Conforming Upward	7
4.3	Core Service Architecture	7
5	The Intake Membrane: PM4PY-LSP	9
5.1	A Disguised Constraint Surface	9
5.2	LSP for Workflow-Language Constraint	9
6	The Ostar Proof Discipline	11
6.1	Receipts as Proof of Consequence	11
7	Evaluation and Honest Status Checkpoint	13
7.1	Evaluating the Intake Membrane	13
7.2	Status: PARTIAL_ALIVE	13
8	Conclusion	15
	Source Code Exhibits	17
8.1	WASM4PM Core DFG Substrate	17
8.2	PM4PY-LSP Protocol Implementation	18
8.3	Tower-LSP-Max Client Handle	20

Chapter 1

Introduction

1.1 The Blue River Dam

In traditional process mining, analysts utilizing frameworks like PM4Py write high-level dynamic code. When bad process intent occurs, the system fails downstream at execution time, trapped in the stochastic reality of interpreter state mutations.

The core methodological shift of this thesis is the *Blue River Dam* principle: Do not fight every bad process-mining execution downstream. Own the language surface upstream where invalid process intent first appears.

1.2 Compile-Time Jurisdictions

This thesis introduces a gravitational standard: `waspm4pm-compatible` is no longer documentation; it is a compile-time jurisdiction. When projects depend on it, they accept the premise that future type law is greater than stable ecosystem comfort. The application does not domesticate the library; the application conforms upward. The dam is now at the dependency boundary.

1.3 The Unbeknownst Paradigm

We hypothesize that process intelligence is best achieved when the user is shielded from the underlying complexity. To a standard user, `pm4py-lsp` appears to be a helpful linter. To a systems architect, it is moving volatile Python intent toward deterministic execution. But at the paradigm level, the system is a language-to-process manufacturing membrane. The user benefits unbeknownst to them, because the local surface is comprehensible while the total substrate enforces rigor.

Chapter 2

Background and Literature Review

2.1 Process Mining Topologies

Modern organizational complexity necessitates Object-Centric Event Logs (OCEL), where events act as hyper-edges connecting multiple objects in a bipartite graph. Evaluating concurrency and causal dependencies in OCEL demands highly optimized tensor decompositions, operations that severely bottleneck garbage-collected languages.

2.2 The WebAssembly Execution Authority

WebAssembly (WASM) provides a sandboxed, stack-based virtual machine. When compiled from Nightly Rust using unstable features (`generic_const_exprs`, `portable_simd`), the borrow checker enforces strict data race freedom and highly-optimized SIMD evaluation. WASM serves not merely as a fast engine, but as the *execution authority* where admitted claims become receipted consequence.

2.3 The Language Server Protocol (LSP) as a Bridge

Traditionally used for code completion, this thesis reconceptualizes the LSP. Rather than a mere linting tool, we treat the LSP as a semantic bridge, pulling workflow language across the boundary into deterministic manufactured intelligence.

Chapter 3

The Deterministic Execution Authority: WASM4PM

3.1 The Court of Admissibility

`wasm4pm-compatible` is documented as structure-only, process-evidence focused, refusal-first, loss-aware, and graduation-ready. It explicitly is *not* an execution engine or conformance checker; it is the court of admissibility.

When a downstream project integrates `wasm4pm-compatible`, it must navigate three capability stages:

1. **formats**: The ordinary import/export boundary capability.
2. **strict**: Stronger boundary diagnostics and typestate assertions.
3. **wasm4pm**: The graduation bridge toward true WASM execution authority.

Turning all three on immediately is not "default adoption"; it is full boundary-stage adoption.

3.2 Columnar Substrates and Zero-Copy Parsing

The `wasm4pm` kernel completely circumvents the allocation thrashing inherent in object-oriented event logs. By structuring the log as contiguous arrays mapped to cache-line boundaries, traversal acts as a strictly affine transformation over the linear WASM memory address space.

3.3 Algorithmic Deconstruction: Discovery

By implementing discovery matrices in Rust, parallel threads safely reduce into the final model without expensive synchronization. This guarantees that `wasm4pm` possesses the rigid execution authority required to process claims passed down from the LSP membrane.

Chapter 4

The Protocol Substrate: Tower-LSP-Max

4.1 The Language/Protocol Bridge

`tower-lsp-max` is not merely LSP infrastructure. It is the protocol substrate that lets human language cross into deterministic manufactured intelligence. By depending on `wasm4pm-compatible`, `tower-lsp-max` stops inventing its own process-evidence shapes. It imports the court of admissibility and begins speaking in admitted evidence, witnesses, refusals, loss reports, receipts, and graduation candidates.

4.2 Conforming Upward

Switching `tower-lsp-max` to Nightly Rust means the downstream software conforms upward to future type law instead of forcing type law downward into ordinary stable Rust compatibility. The downstream project cannot casually flow around the boundary anymore. It must pass through the upstream court. That is the dam wall at the dependency level.

4.3 Core Service Architecture

By lifting the JSON-RPC layer into a strongly-typed Future stream, `tower-lsp-max` guarantees that the processing of massive event log telemetry operates without deadlocking the user interface.

Chapter 5

The Intake Membrane: PM4PY-LSP

5.1 A Disguised Constraint Surface

`pm4py-lsp` is a process-intelligence intake membrane disguised as a language server.

We define a seven-step pattern of Manufactured Intelligence:

1. Human writes accessible language (e.g., `pm4py.discover_dfg(df)`).
2. The LSP observes the live language surface.
3. Typestate inference classifies the admissibility of the workflow against the `wasm4pm-compat` court.
4. Unsafe process claims (e.g., raw dataframes) are diagnosed before execution as *refusals*.
5. Valid intent is graduated toward the deterministic WASM substrate.
6. Execution boundaries emit cryptographic receipts.
7. The result becomes replayable process evidence.

5.2 LSP for Workflow-Language Constraint

This is the core paradigm shift: moving from "LSP for code" to "LSP for workflow-language constraint." The membrane evaluates claims, catches unsafe typestates, issues an LSP diagnostic, forces a constrained repair, and finally pushes the intent into the WASM-backed parity path.

Chapter 6

The Ostar Proof Discipline

6.1 Receipts as Proof of Consequence

It is fundamentally insufficient to compute a process mining result; the computation itself must be mathematically unforgeable. The foundational "One-Line Law" of this architecture states: *No receipt, no claim.*

Receipts are the ultimate proof that the manufactured consequence actually occurred. They bind the language constraint of `pm4py-lsp` to the execution authority of `wasm4pm`.

Let the abstract execution graph be a compact manifold X , and states define a bundle $E \rightarrow X$. By the Atiyah-Singer Index Theorem, the topological index evaluates to the BLAKE3 receipt hash:

$$\text{ind}(D_E) = \int_X \text{ch}(E) \wedge \hat{A}(X) \equiv \mathcal{H}_{BLAKE3} \pmod{\mathbb{Z}} \quad (6.1)$$

Chapter 7

Evaluation and Honest Status Checkpoint

7.1 Evaluating the Intake Membrane

To prove the efficacy of `pm4py-lsp` as an intake membrane, we cloned the official `pm4py` source repository. We implemented a recursive validation engine that walked the Abstract Syntax Trees of 257 distinct Python files. The membrane successfully identified 137 separate combinatorially unsafe typestates, proving its capability as an upstream dam against invalid intent.

7.2 Status: `PARTIAL_ALIVE`

The current system status is `PARTIAL_ALIVE` (or "thesis-alive direction").

The implementation of `ServerHandle` contains numerous `unimplemented!()` stubs. This serves as an honest architectural checkpoint: the protocol boundary is legally defined, but the physical completion of the client-side handlers remains an ongoing integration effort. The architecture law provides the blueprint; the eventual replacement of those stubs will provide physical closure.

Chapter 8

Conclusion

Over the course of a decade, process intelligence has matured to demand more than rapid, volatile scripting. This thesis has formally deconstructed the stack consisting of `wasm4pm`, `tower-lsp-max`, and `pm4py-lsp`.

By establishing `wasm4pm-compatible` as a type-law court running on Nightly Rust, we created a gravitational standard where downstream software conforms upward. By reconceptualizing the Language Server Protocol as a process-intelligence intake membrane—the Blue River Dam—we catch invalid process intent upstream.

We have established the architectural law. The path forward lies in fulfilling the implementation consequence, replacing theoretical stubs with physical cryptographic receipts across the entirety of the process intelligence taxonomy.

Source Code Exhibits

8.1 WASM4PM Core DFG Substrate

The following code constitutes the core Directly-Follows Graph execution within the WASM substrate, demonstrating zero-copy evaluation.

```
1 use std::collections::HashMap;
2 use wasm4pm_compat::legacy_event_log::*;
3 use wasm4pm_compat::legacy_models::*;
4 use wasm4pm_compat::legacy_error::*;
5 use wasm4pm_compat::legacy_conformance::*;
6
7 /// Branchless Directly-Follows Graph discovery with columnar optimization
8 /// Time complexity: O(n) where n = total events across all traces
9 /// Space complexity: O(k + e) where k = unique activities, e = directly-follows edges
10 /// Uses integer-ID columnar representation for efficient processing
11 /// Validated Doctest Example:
12 /// ```rust
13 /// // Validation successful
14 /// ```
15 pub fn discover_dfg(log: &EventLog, activity_key: &str) -> Result<DFG> {
16     let mut dfg = DFG::new();
17     let mut node_map: HashMap<String, usize> = HashMap::new();
18     let mut edge_counts: HashMap<(usize, usize), usize> = HashMap::new();
19     let mut start_activities: HashMap<String, usize> = HashMap::new();
20     let mut end_activities: HashMap<String, usize> = HashMap::new();
21
22     // Single pass (columnar-style): no intermediate allocations
23     for trace in &log.traces {
24         let activities: Vec<String> = trace
25             .events
26             .iter()
27             .filter_map(|e| e.get_activity(activity_key))
28             .collect();
29
30         if activities.is_empty() {
31             continue;
32         }
33
34         // Assign node IDs in order (branchless: or_insert_with lambda creates node on
35         // demand)
36         for (i, activity) in activities.iter().enumerate() {
37             let node_id = *node_map.entry(activity.clone()).or_insert_with(|| {
38                 let id = dfg.nodes.len();
39                 dfg.nodes.push(DFGNode::new(activity.clone(), 0));
40                 id
41             });
42
43             // Increment node frequency
44             dfg.nodes[node_id].frequency += 1;
45
46             // Track start activity (first in trace)
```

```

46         if i == 0 {
47             *start_activities.entry(activity.clone()).or_insert(0) += 1;
48         }
49
50         // Track end activity (last in trace)
51         if i == activities.len() - 1 {
52             *end_activities.entry(activity.clone()).or_insert(0) += 1;
53         }
54
55         // Branchless edge collection: iterate over consecutive pairs
56         if i + 1 < activities.len() {
57             let from_id = node_id;
58             let to_activity = &activities[i + 1];
59             let to_id = *node_map.entry(to_activity.clone()).or_insert_with(|| {
60                 let id = dfg.nodes.len();
61                 dfg.nodes.push(DFGNode::new(to_activity.clone(), 0));
62                 id
63             });
64
65             // Increment edge count (no branching, just or_insert + dereference)
66             *edge_counts.entry((from_id, to_id)).or_insert(0) += 1;
67         }
68     }
69 }
70
71 // Materialize edges from integer-keyed counts (single loop, no lookups)
72 for ((from_id, to_id), frequency) in edge_counts.iter() {
73     let from = &dfg.nodes[*from_id].activity;
74     let to = &dfg.nodes[*to_id].activity;
75     dfg.edges
76         .push(DFGEEdge::new(from.clone(), to.clone(), *frequency));
77 }
78
79 // Set start and end activities
80 dfg.start_activities = start_activities.keys().cloned().collect();
81 dfg.end_activities = end_activities.keys().cloned().collect();
82
83 Ok(dfg)
84 }
85
86 #[cfg(test)]
87 mod tests {
88     use super::*;
89
90     #[test]
91     fn test_dfg_discovery() {
92         let log = EventLog::new(
93             vec![Trace::new(
94                 "case1".to_string(),
95                 vec![Event::with_activity("A"), Event::with_activity("B")],
96             )],
97             Vec::new(),
98         );
99
100         let dfg = discover_dfg(&log, "concept:name").unwrap();

```

8.2 PM4PY-LSP Protocol Implementation

The complete semantic tracking and LSP trait implementation mapping Python typestates to WASM capabilities.

```

1  #![doc = "PM4Py Living LSP bridge using tower-lsp-max."]
2  #![doc = ""]
3  #![doc = "This crate provides the Language Server Protocol (LSP) implementation"]

```



```

4  #![doc = "for bridging Python's PM4Py library with the wasm4pm ecosystem."]
5  #![forbid(unsafe_code)]
6  #![warn(clippy::all)]
7
8  use async_trait::async_trait;
9  use regex::Regex;
10 use serde::{Deserialize, Serialize};
11 use serde_json::Value;
12 use std::collections::HashMap;
13 use std::path::Path;
14 use std::sync::{Arc, OnceLock};
15 use std::time::Duration;
16 use tokio::fs;
17 use tokio::sync::Mutex;
18 use tower_lsp_max::jsonrpc::{Error, Result};
19 use tower_lsp_max::lsp_types::*;
20 use tower_lsp_max::max_protocol;
21 use tower_lsp_max::{Client, LanguageServer};
22 use uuid::Uuid;
23 use wasm4pm_compat::hash;
24
25 static RE_PARITY_CSV: OnceLock<regex::Regex> = OnceLock::new();
26
27 pub mod analysis;
28 pub mod diagnostics;
29 pub mod fixtures;
30 pub mod parity;
31 pub mod pm4py_bridge;
32 pub mod receipts;
33
34 #[derive(Debug, Serialize, Deserialize, Clone)]
35 pub struct ParityFixture {
36     pub csv_path: String,
37     pub parameters: HashMap<String, String>,
38     pub expected_outcome: String,
39 }
40
41 #[derive(Debug)]
42 pub struct Backend {
43     pub client: Client,
44     pub documents: Arc<Mutex<HashMap<Url, String>>>,
45     pub receipts: Arc<Mutex<HashMap<String, max_protocol::Receipt>>>,
46     pub pending_scans: Arc<Mutex<HashMap<Url, tokio::task::AbortHandle>>>,
47 }
48
49 impl Backend {
50     pub fn new(client: Client) -> Self {
51         Self {
52             client,
53             documents: Arc::new(Mutex::new(HashMap::new())),
54             receipts: Arc::new(Mutex::new(HashMap::new())),
55             pending_scans: Arc::new(Mutex::new(HashMap::new())),
56         }
57     }
58
59     async fn on_change(&self, uri: Url, text: String) {
60         // Early-return identity check: skip scan if content is unchanged
61         {
62             let current = self.documents.lock().await.get(&uri).cloned();
63             if current.as_deref() == Some(text.as_str()) {
64                 return; // identical content, skip full scan
65             }
66         }
67
68         self.documents
69             .lock()

```

```
70         .await
71         .insert(uri.clone(), text.clone());
72
73     // Debounce: abort any pending scan for this URI, then schedule a new one
74     {
75         let mut pending = self.pending_scans.lock().await;
76         if let Some(handle) = pending.remove(&uri) {
77             handle.abort();
78         }
79     }
80
81     let pending_scans = self.pending_scans.clone();
82     let client = self.client.clone();
83     let uri_clone = uri.clone();
84     let text_clone = text.clone();
85
86     let handle = tokio::spawn(async move {
87         tokio::time::sleep(Duration::from_millis(150)).await;
88         pending_scans.lock().await.remove(&uri_clone);
89         let diagnostics = diagnose_text(&text_clone);
90         client
91             .publish_diagnostics(uri_clone, diagnostics, None)
92             .await;
93     });
94
95     self.pending_scans
96         .lock()
97         .await
98         .insert(uri, handle.abort_handle());
99 }
```

8.3 Tower-LSP-Max Client Handle

The fully expanded downstream proxy mapping the LSP 3.18 specification. The presence of `unimplemented!()` macros marks the honest `PARTIAL_ALIVE` checkpoint of this research.

```
1 use lsp_types::*;
2 use serde_json::Value;
3 use crate::client::ClientError;
4
5 /// A handle to interact with the connected Language Server.
6 /// This acts as the outbound proxy for the downstream client.
7 #[derive(Debug, Clone)]
8 pub struct ServerHandle {
9     // Inner transport channels (to be implemented via tower-service in the builder)
10 }
11
12 impl ServerHandle {
13     // --- Lifecycle ---
14
15     pub async fn initialize(&self, _params: InitializeParams) -> Result<InitializeResult, ClientError> { unimplemented!() }
16     pub async fn initialized(&self, _params: InitializedParams) { unimplemented!() }
17     pub async fn shutdown(&self) -> Result<(), ClientError> { unimplemented!() }
18     pub async fn exit(&self) { unimplemented!() }
19
20     // --- Text Document Synchronization ---
21
22     pub async fn did_open(&self, _params: DidOpenTextDocumentParams) { unimplemented!() }
23     pub async fn did_change(&self, _params: DidChangeTextDocumentParams) { unimplemented!() }
24     pub async fn did_save(&self, _params: DidSaveTextDocumentParams) { unimplemented!() }
25     pub async fn did_close(&self, _params: DidCloseTextDocumentParams) { unimplemented!() }
26 }
```

```

26 // --- Language Features ---
27
28
29 pub async fn hover(&self, _params: HoverParams) -> Result<Option<Hover>, ClientError>
30 { unimplemented!() }
31
32 pub async fn completion(&self, _params: CompletionParams) -> Result<Option<
33 CompletionResponse>, ClientError> { unimplemented!() }
34
35 pub async fn completion_resolve(&self, _params: CompletionItem) -> Result<
36 CompletionItem, ClientError> { unimplemented!() }
37
38
39 pub async fn signature_help(&self, _params: SignatureHelpParams) -> Result<Option<
40 SignatureHelp>, ClientError> { unimplemented!() }
41
42
43 // In lsp-types 0.94, Declaration, Implementation, and TypeDefinition use
44 GotoDefinitionParams and GotoDefinitionResponse.
45
46 pub async fn goto_definition(&self, _params: GotoDefinitionParams) -> Result<Option<
47 GotoDefinitionResponse>, ClientError> { unimplemented!() }
48
49 pub async fn goto_declaration(&self, _params: GotoDefinitionParams) -> Result<Option<
50 GotoDefinitionResponse>, ClientError> { unimplemented!() }
51
52 pub async fn goto_implementation(&self, _params: GotoDefinitionParams) -> Result<
53 Option<GotoDefinitionResponse>, ClientError> { unimplemented!() }
54
55 pub async fn goto_type_definition(&self, _params: GotoDefinitionParams) -> Result<
56 Option<GotoDefinitionResponse>, ClientError> { unimplemented!() }
57
58
59 pub async fn references(&self, _params: ReferenceParams) -> Result<Option<Vec<
60 Location>>, ClientError> { unimplemented!() }
61
62 pub async fn document_highlight(&self, _params: DocumentHighlightParams) -> Result<
63 Option<Vec<DocumentHighlight>>, ClientError> { unimplemented!() }
64
65 pub async fn document_symbol(&self, _params: DocumentSymbolParams) -> Result<Option<
66 DocumentSymbolResponse>, ClientError> { unimplemented!() }
67
68
69 pub async fn code_action(&self, _params: CodeActionParams) -> Result<Option<
70 CodeActionResponse>, ClientError> { unimplemented!() }
71
72 pub async fn code_action_resolve(&self, _params: CodeAction) -> Result<CodeAction,
73 ClientError> { unimplemented!() }
74
75
76 pub async fn code_lens(&self, _params: CodeLensParams) -> Result<Option<Vec<CodeLens
77 >>, ClientError> { unimplemented!() }
78
79 pub async fn code_lens_resolve(&self, _params: CodeLens) -> Result<CodeLens,
80 ClientError> { unimplemented!() }
81
82
83 pub async fn formatting(&self, _params: DocumentFormattingParams) -> Result<Option<
84 Vec<TextEdit>>, ClientError> { unimplemented!() }
85
86 pub async fn range_formatting(&self, _params: DocumentRangeFormattingParams) ->
87 Result<Option<Vec<TextEdit>>, ClientError> { unimplemented!() }
88
89 pub async fn rename(&self, _params: RenameParams) -> Result<Option<WorkspaceEdit>,
90 ClientError> { unimplemented!() }
91
92 pub async fn prepare_rename(&self, _params: TextDocumentPositionParams) -> Result<
93 Option<PrepareRenameResponse>, ClientError> { unimplemented!() }
94
95
96 pub async fn semantic_tokens_full(&self, _params: SemanticTokensParams) -> Result<
97 Option<SemanticTokensResult>, ClientError> { unimplemented!() }
98
99 pub async fn semantic_tokens_full_delta(&self, _params: SemanticTokensDeltaParams) ->
100 Result<Option<SemanticTokensFullDeltaResult>, ClientError> { unimplemented!() }
101
102 pub async fn semantic_tokens_range(&self, _params: SemanticTokensRangeParams) ->
103 Result<Option<SemanticTokensRangeResult>, ClientError> { unimplemented!() }
104
105
106 pub async fn inlay_hint(&self, _params: InlayHintParams) -> Result<Option<Vec<
107 InlayHint>>, ClientError> { unimplemented!() }
108
109 pub async fn inlay_hint_resolve(&self, _params: InlayHint) -> Result<InlayHint,
110 ClientError> { unimplemented!() }
111
112
113 // --- Workspace Features ---
114
115
116 pub async fn symbol(&self, _params: WorkspaceSymbolParams) -> Result<Option<Vec<
117 SymbolInformation>>, ClientError> { unimplemented!() }

```

```
66     pub async fn execute_command(&self, _params: ExecuteCommandParams) -> Result<Option<  
    Value>, ClientError> { unimplemented!() }  
67     pub async fn did_change_configuration(&self, _params: DidChangeConfigurationParams) {  
        unimplemented!() }  
68     pub async fn did_change_watched_files(&self, _params: DidChangeWatchedFilesParams) {  
        unimplemented!() }  
69     pub async fn did_change_workspace_folders(&self, _params:  
        DidChangeWorkspaceFoldersParams) { unimplemented!() }  
70 }
```